

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

Факультет информационных технологий
Кафедра «Инфокогнитивные технологии»

КУРСОВОЙ ПРОЕКТ

по дисциплине «Разработка мобильных приложений»

на тему: «Разработка приложения «Road Editor» для проектирования
дорожных карт»

Направление подготовки 09.03.03 «Прикладная информатика»
Профиль «Корпоративные информационные системы»

Выполнила:
студентка группы 231-361
Чарыкова Наталия Олеговна

Москва 2026

Введение

Современные программные средства визуального проектирования позволяют ускорить подготовку схем, макетов и учебных моделей. Одной из задач такого типа является построение дорожных карт, включающих прямые участки, повороты, перекрестки, пешеходные переходы и дополнительные графические обозначения. При выполнении этой задачи вручную пользователь вынужден многократно копировать однотипные элементы, выравнивать их и контролировать взаимное расположение. Это увеличивает время работы и повышает вероятность ошибок.

В рамках дисциплины «Разработка мобильных приложений» разработано графическое приложение Road Editor, предназначенное для конструирования дорожных карт на клеточном поле. Приложение позволяет выбирать готовые дорожные элементы из палитры, размещать их в ячейках карты, изменять размер рабочей области, заполнять карту выбранным типом элемента и дополнять схему произвольными графическими фигурами.

Практическая ценность проекта заключается в объединении двух способов редактирования. Первый способ основан на размещении готовых плиток, каждая из которых соответствует определенному фрагменту дорожной инфраструктуры. Второй способ предусматривает рисование линий, прямоугольников, эллипсов, треугольников, стрелок и звезд поверх карты. Благодаря этому приложение подходит как для быстрого построения дорожной сети, так и для добавления поясняющих обозначений.

В приложении реализованы сохранение и загрузка проекта, экспорт результата в формат PNG, отмена и повтор действий, масштабирование, перемещение рабочей области, выбор цветовой схемы и автоматическое сохранение пользовательских настроек. Программный код разделен на проект пользовательского интерфейса и библиотеку основной логики, что упрощает сопровождение и повторное использование компонентов.

1 Цель и задачи работы

Целью работы является разработка приложения Road Editor, обеспечивающего создание, редактирование, сохранение и экспорт

дорожных карт с использованием готовых дорожных элементов и дополнительных графических фигур.

Для достижения поставленной цели необходимо решить следующие задачи:

1. определить функциональные требования и основные пользовательские сценарии;
2. спроектировать структуру приложения с разделением интерфейса и основной логики;
3. разработать модель дорожной карты и перечень поддерживаемых дорожных элементов;
4. реализовать палитру прямых дорог, поворотов, развилок, перекрестков и тротуарной плитки;
5. реализовать изменение размера карты с сохранением ранее размещенных элементов;
6. реализовать рисование, выделение, перемещение и удаление дополнительных фигур;
7. реализовать отмену и повтор действий;
8. реализовать масштабирование и перемещение рабочей области;
9. реализовать сохранение и загрузку проектов в формате `.roadmap`;
10. реализовать экспорт готовой схемы в формат PNG;
11. реализовать справочную систему и пользовательские настройки;
12. подготовить приложение к демонстрации и дальнейшему расширению.

В качестве языка программирования используется C#, а пользовательский интерфейс реализован средствами Windows Presentation Foundation. Проект ориентирован на операционную систему Windows и использует возможности платформы .NET. Для сериализации проектов и пользовательских настроек применяется формат JSON.

2 Проектирование приложения

2.1 Функциональные требования

К основным функциональным требованиям относятся создание карты заданного размера, размещение дорожных элементов, изменение существующей карты, работа с графическими фигурами, сохранение промежуточного результата и экспорт итогового изображения.

Пользователь должен иметь возможность выполнять следующие действия:

- задавать ширину и высоту карты;
- выбирать элемент в палитре;
- размещать выбранный элемент в ячейке;
- использовать пустую ячейку и полностью заполненную тротуарной плиткой ячейку;
- заполнять всю карту выбранным элементом;
- переключаться между режимами размещения дорог, выбора и рисования;
- выбирать цвет контура и заливки фигуры;
- сохранять проект и восстанавливать его из файла;
- экспортировать карту вместе с фигурами в PNG;
- изменять масштаб и перемещать рабочую область;
- использовать сочетания клавиш для отмены, повтора и вызова справки.

2.2 Архитектура приложения

Приложение построено по модульному принципу. Проект `RoadEditor.App` содержит окно приложения, XAML-разметку, обработчики пользовательских действий, отрисовку дорожных элементов, работу с фигурами, масштабирование и операции сохранения. Проект `RoadEditor.Core` содержит модель дорожной карты и классы пользовательских настроек.

На рисунке 1 представлена общая архитектура приложения.

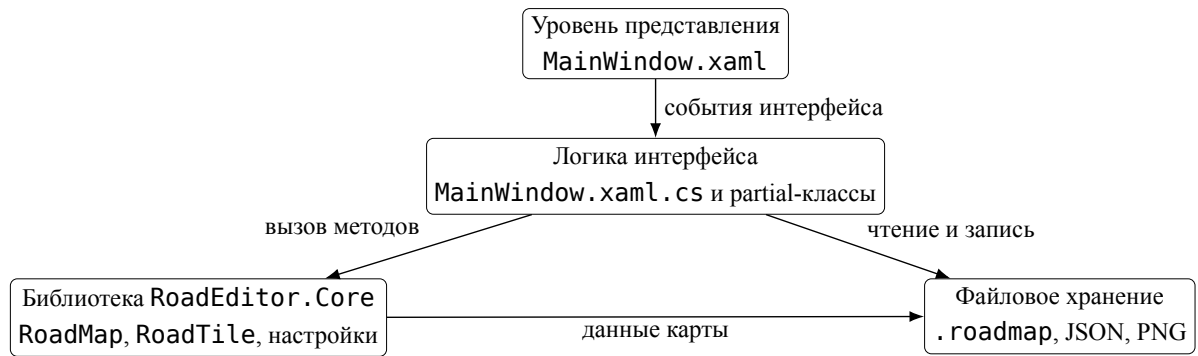


Рис. 1 – Архитектура приложения Road Editor

Такое разделение уменьшает связанность компонентов. Модель карты не зависит от визуальных элементов WPF и может использоваться в другом типе интерфейса. Обработчики окна получают данные из модели, выполняют отрисовку и передают изменения обратно в библиотеку основной логики.

2.3 Структура классов

Центральной сущностью основной логики является класс **RoadMap**. Он хранит двумерный массив объектов **RoadTile**, предоставляет индексатор и методы изменения карты. Интерфейс **IRoadMap** описывает доступные операции и событие изменения данных. Абстрактный класс **MapElement** задает координаты элемента, а **RoadTile** наследует эти свойства и хранит тип плитки.

На рисунке 2 показаны основные классы приложения и связи между ними.

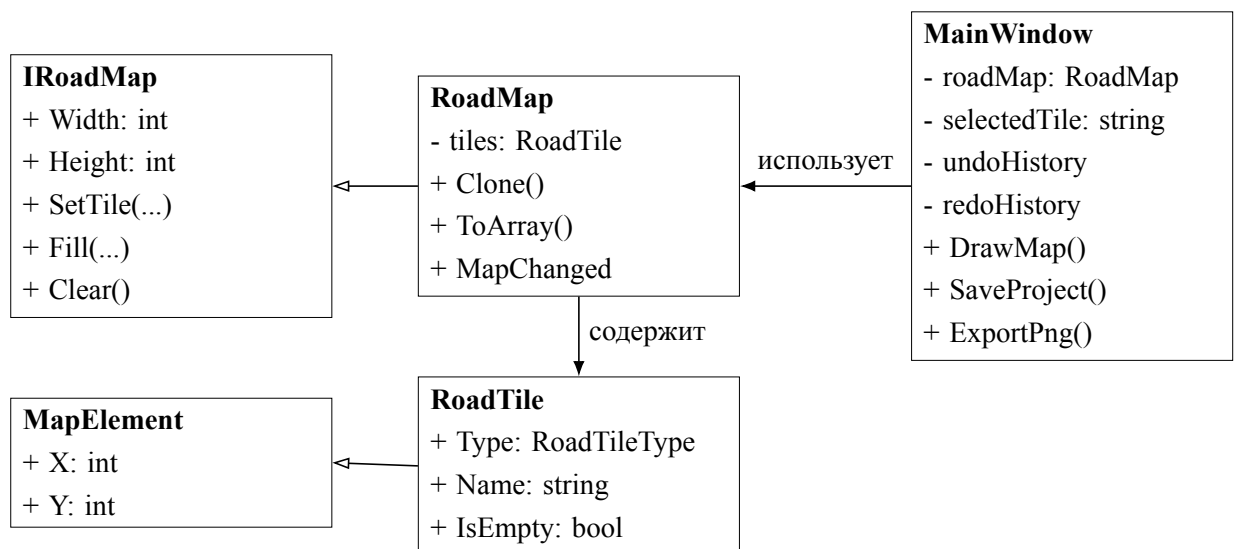


Рис. 2 – Диаграмма основных классов приложения

2.4 Варианты использования

Основным актором системы является пользователь. После запуска приложения ему доступны все функции редактора. Авторизация и серверная часть не требуются, поскольку проект является локальным графическим приложением.



Рис. 3 – Диаграмма вариантов использования

2.5 Структура сохраняемого проекта

Файл проекта имеет расширение **.roadmap** и содержит данные в формате JSON. В файл включаются ширина и высота карты, двумерный массив дорожных элементов и сведения о дополнительных фигурах. Такое представление является текстовым, переносимым и удобным для отладки.

Таблица 1 – Основные данные проекта

Поле	Назначение
Width	Ширина карты в ячейках
Height	Высота карты в ячейках
Tiles	Двумерный массив названий дорожных элементов
Shapes	Список дополнительных графических фигур и их параметров

2.6 Проектирование интерфейса

Интерфейс разделен на три основные области. В верхней части располагается меню с командами отмены, повтора, очистки, сохранения, загрузки, экспорта и вызова справки. Слева находится панель управления, содержащая параметры карты, режимы рисования, выбор цветов, просмотр текущего дорожного элемента, масштабирование, настройки и палитру. Справа располагается рабочая область карты.

В приложении используются темная рабочая область и контрастная боковая панель. Для выделения выбранного элемента применяется голубая рамка. Дорожные плитки имеют текстуру асфальта и тротуарной кладки, что позволяет визуально отличать проезжую часть от окружающего покрытия.

3 Реализация приложения

3.1 Средства разработки

Проект реализован на языке C# для платформы .NET. Интерфейс окна описан на языке XAML, а обработка событий и отрисовка выполняются в коде C#. Для построения фигур используются классы пространства имен `System.Windows.Shapes`. Сохранение данных выполняется средствами `System.Text.Json`, а экспорт изображения основан на классе `RenderTargetBitmap`.

Решение состоит из двух проектов:

- `RoadEditor.App` – WPF-приложение и пользовательский интерфейс;
- `RoadEditor.Core` – библиотека классов с моделью карты и настройками.

3.2 Модель дорожной карты

Перечисление `RoadTileType` содержит все поддерживаемые плитки. Помимо пустой ячейки и дорожных участков в него включена полностью кирпичная плитка `Pavement`. Фрагмент модели приведен в листинге 1.

```
1 public enum RoadTileType
2 {
3     Empty,
4     Pavement,
```

```

5      RoadHorizontal,
6      RoadVertical,
7      CornerRightDown,
8      CornerLeftDown,
9      CornerRightUp,
10     CornerLeftUp,
11     TUp,
12     TDown,
13     TLeft,
14     TRight,
15     Cross
16 }
17
18 public sealed class RoadTile : MapElement
19 {
20     public RoadTileType Type { get; private set; }
21     public string Name => Type.ToString();
22     public bool IsEmpty => Type == RoadTileType.Empty;
23 }

```

Листинг 1 – Модель типов дорожных элементов

Класс **RoadMap** хранит элементы в двумерном массиве. Перед обращением к элементу выполняется проверка координат. Методы **SetTile**, **Fill** и **Clear** изменяют состояние карты и создают событие **MapChanged**. Наличие интерфейса **IRoadMap** позволяет использовать модель независимо от конкретного окна приложения.

3.3 Формирование палитры и отрисовка плиток

Палитра формируется программно на основе массива строковых идентификаторов. Каждый элемент получает фон, графическое содержимое и рамку. Выбранная плитка выделяется голубым цветом.

Для полностью кирпичной ячейки сначала вызывается метод **DrawPavement**, после чего дополнительная дорога не рисуется. Логика обработки показана в листинге 2.

```

1 private readonly string[] palette =
2 {
3     "Empty",
4     "Pavement",
5     "RoadHorizontal",

```



```

6      "RoadVertical",
7      "CornerRightDown",
8      "CornerLeftDown",
9      "CornerRightUp",
10     "CornerLeftUp",
11     "TUp",
12     "TDown",
13     "TLeft",
14     "TRight",
15     "Cross"
16 };
17
18 private void DrawTile(Canvas canvas, string tile,
19                       double x, double y, double size)
20 {
21     if (tile == "Empty")
22         return;
23
24     DrawPavement(canvas, x, y, size);
25
26     switch (tile)
27     {
28         case "Pavement":
29             break;
30         case "RoadHorizontal":
31             DrawRoad(canvas, x, y, size,
32                     left: true, right: true,
33                     up: false, down: false);
34             break;
35     }
36 }

```

Листинг 2 – Обработка полностью кирпичной плитки

Тротуарная поверхность строится из прямоугольника основного цвета, горизонтальных и вертикальных швов, а также небольших светлых и темных отметок. Дорожное покрытие формируется отдельными прямоугольниками и дополняется зернистостью, трещинами и штриховой разметкой.

3.4 Работа с графическими фигурами

Поверх карты располагается отдельный прозрачный холст `DrawingCanvas`. Он используется для рисования линий, прямоугольников, эллипсов, треугольников, стрелок и звезд. Разделение дорожной карты и слоя фигур позволяет независимо редактировать оба типа содержимого.

При выборе режима рисования нажатие мыши определяет начальную точку, перемещение мыши изменяет геометрию предварительной фигуры, а отпускание кнопки завершает действие. В режиме выбора пользователь может выделить фигуру и переместить ее. Для выделения применяется визуальный эффект свечения.

Цвет контура и цвет заливки выбираются в отдельных списках. Флажок «Использовать заливку» позволяет создавать как заполненные, так и контурные фигуры. Выбранный объект можно удалить отдельно, а все фигуры можно очистить одной командой.

3.5 Отмена и повтор действий

Для отмены и повтора используются два стека снимков состояния. Перед изменением карты или фигур текущее состояние помещается в стек отмены, а стек повтора очищается. При выполнении отмены текущее состояние сохраняется в стек повтора, после чего восстанавливается предыдущий снимок.

Команды доступны через пункты меню и сочетания клавиш `Ctrl+Z` и `Ctrl+Y`. Такой подход обеспечивает предсказуемое восстановление как дорожных плиток, так и графических объектов.

3.6 Сохранение, загрузка и экспорт

При сохранении приложение формирует объект данных, сериализует его в JSON и записывает в файл с расширением `.roadmap`. При загрузке выполняется обратная операция: данные считываются, проверяются и используются для восстановления карты и фигур.

Обобщенный алгоритм работы сохранения показан в листинге 3.

```
1 var data = new ProjectSaveData
2 {
3     Width = roadMap.Width,
4     Height = roadMap.Height,
```

```

5     Tiles = roadMap.ToArray(),
6     Shapes = CreateShapeSaveData()
7 };
8
9 string json = JsonSerializer.Serialize(
10     data,
11     new JsonSerializerOptions
12     {
13         WriteIndented = true
14     });
15
16 File.WriteAllText(dialog.FileName, json);

```

Листинг 3 – Сохранение проекта в формате JSON

Экспорт PNG выполняется после измерения и компоновки рабочей области. Объект `RenderTargetBitmap` формирует растровое изображение карты вместе с дополнительными фигурами. Затем данные кодируются с помощью `PngBitmapEncoder` и сохраняются в выбранный файл.

3.7 Масштабирование, перемещение и настройки

Масштаб рабочей области изменяется колесом мыши или кнопками интерфейса. Для преобразования используется `ScaleTransform`. Перемещение большой карты выполняется правой кнопкой мыши через изменение положения полос прокрутки.

Пользователь может выбрать фиолетовую или синюю цветовую схему. Настройки сохраняются в JSON-файл в локальном каталоге приложения и восстанавливаются при следующем запуске. Предусмотрена команда сброса настроек к значениям по умолчанию.

3.8 Алгоритм размещения дорожного элемента

Последовательность обработки щелчка по карте представлена на рисунке 4.

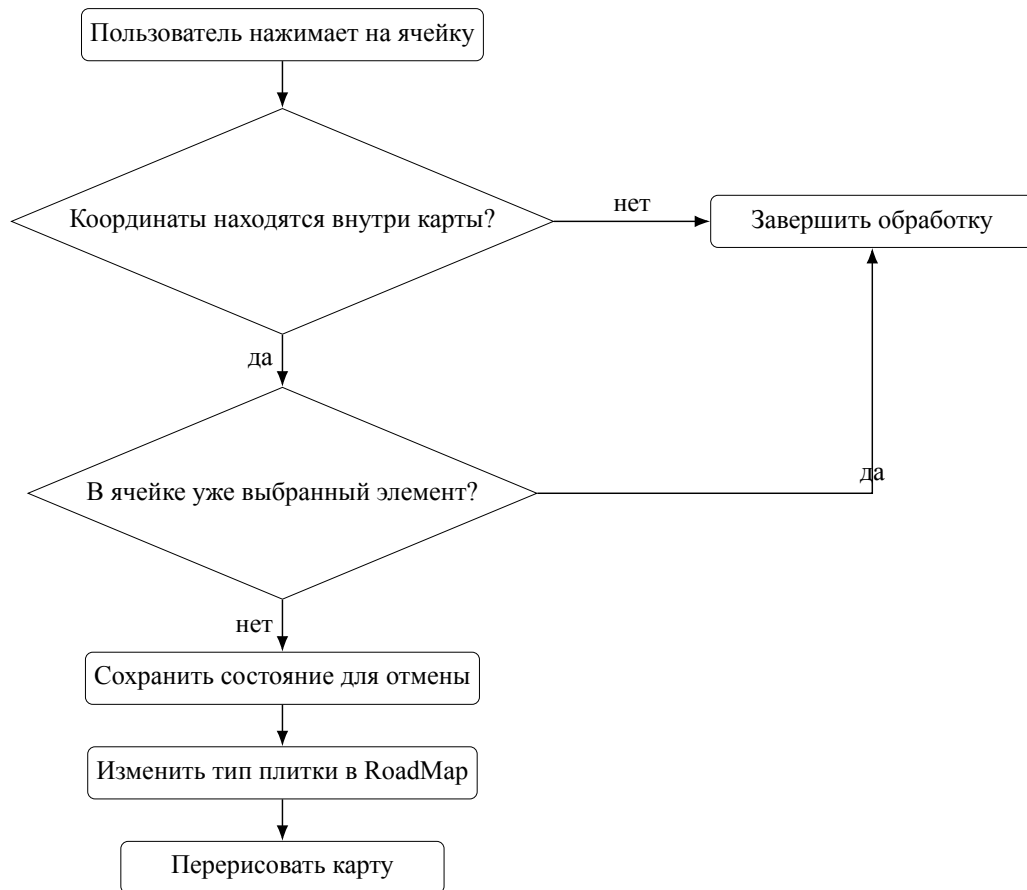


Рис. 4 – Алгоритм размещения дорожного элемента

4 Основные сценарии использования приложения

4.1 Создание дорожной карты

После запуска пользователь задает ширину и высоту карты либо продолжает работу с картой, созданной по умолчанию. В палитре выбирается дорожный элемент, после чего он размещается щелчком по нужной ячейке. На рисунке 5 показано главное окно с картой, заполненной тротуарной плиткой, и несколькими дорожными участками.

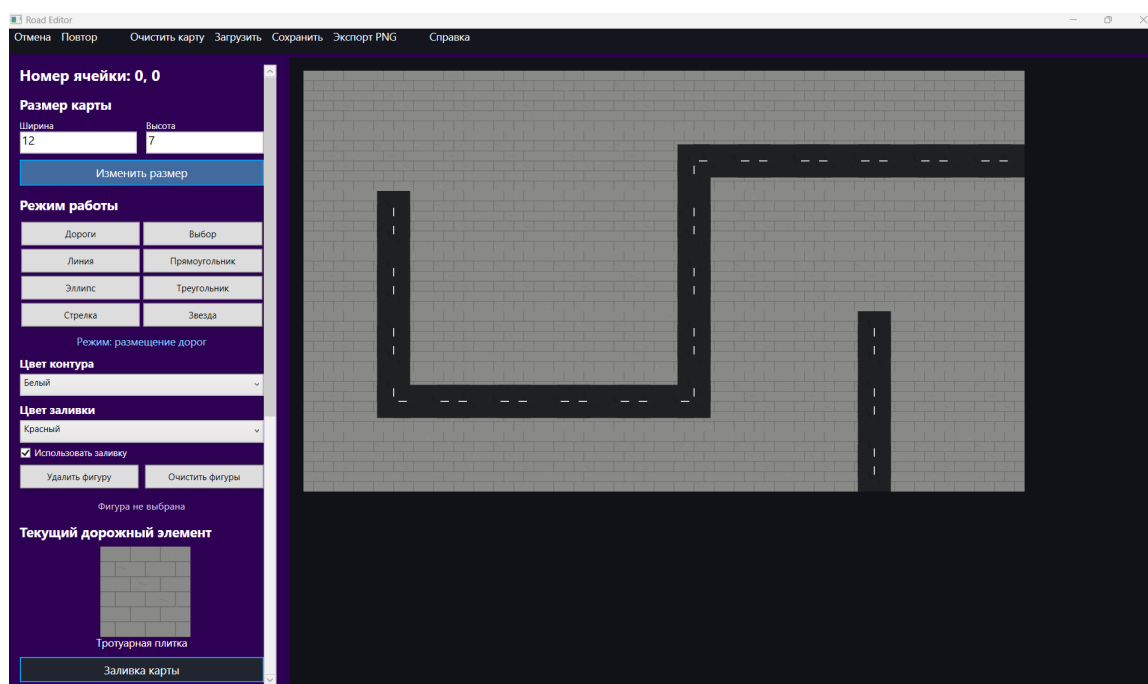


Рис. 5 – Главное окно приложения Road Editor

4.2 Сохранение проекта

Команда «Сохранить» открывает стандартное диалоговое окно. Пользователь выбирает каталог и имя файла. Расширение `.roadmap` подставляется автоматически. Диалог сохранения представлен на рисунке 6.

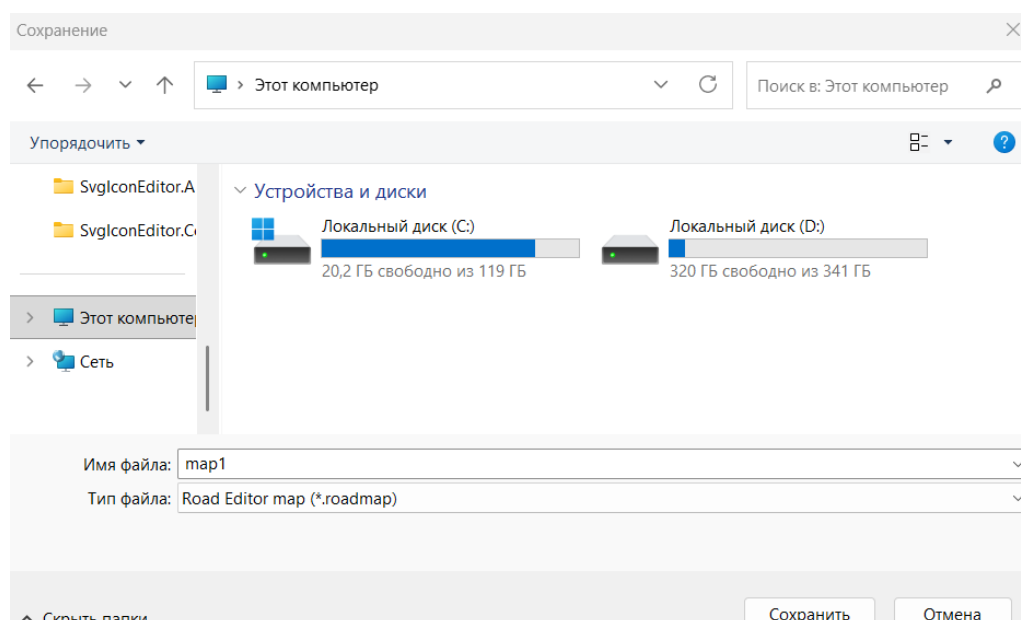


Рис. 6 – Диалог сохранения проекта

4.3 Загрузка проекта

Команда «Загрузить» позволяет выбрать ранее сохраненный файл. После чтения файла приложение восстанавливает размер карты, расположение плиток и дополнительные фигуры. Диалог загрузки показан на рисунке 7.

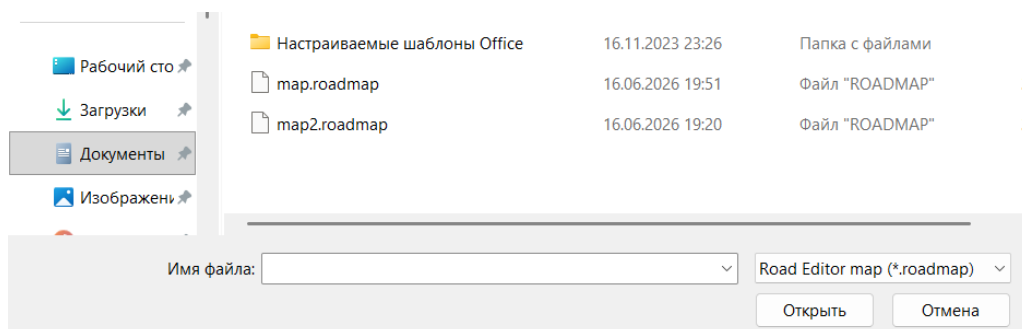


Рис. 7 – Диалог загрузки проекта

4.4 Использование справки

Справка вызывается через пункт меню или клавишу F1. В ней перечислены основные действия и сочетания клавиш. Окно справки представлено на рисунке 8.

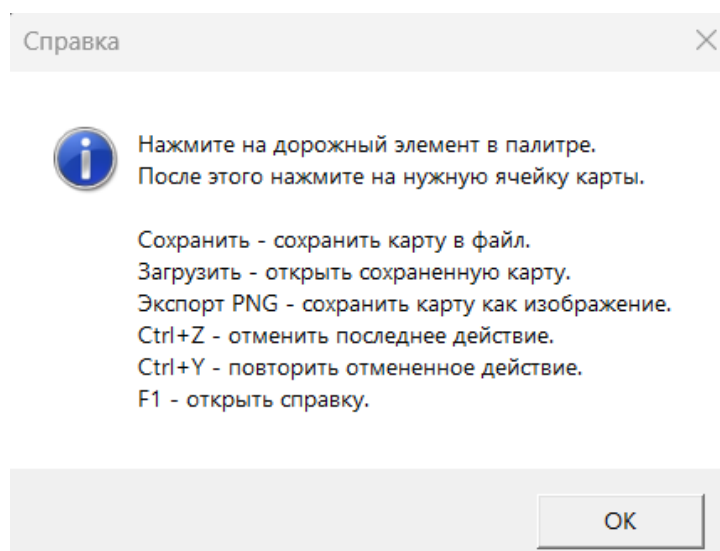


Рис. 8 – Окно справочной информации

4.5 Рисование стрелки

Для добавления указателя пользователь выбирает режим «Стрелка», задает цвет контура и заливки, а затем протягивает мышью по рабочей области. Пример созданной стрелки показан на рисунке 9.

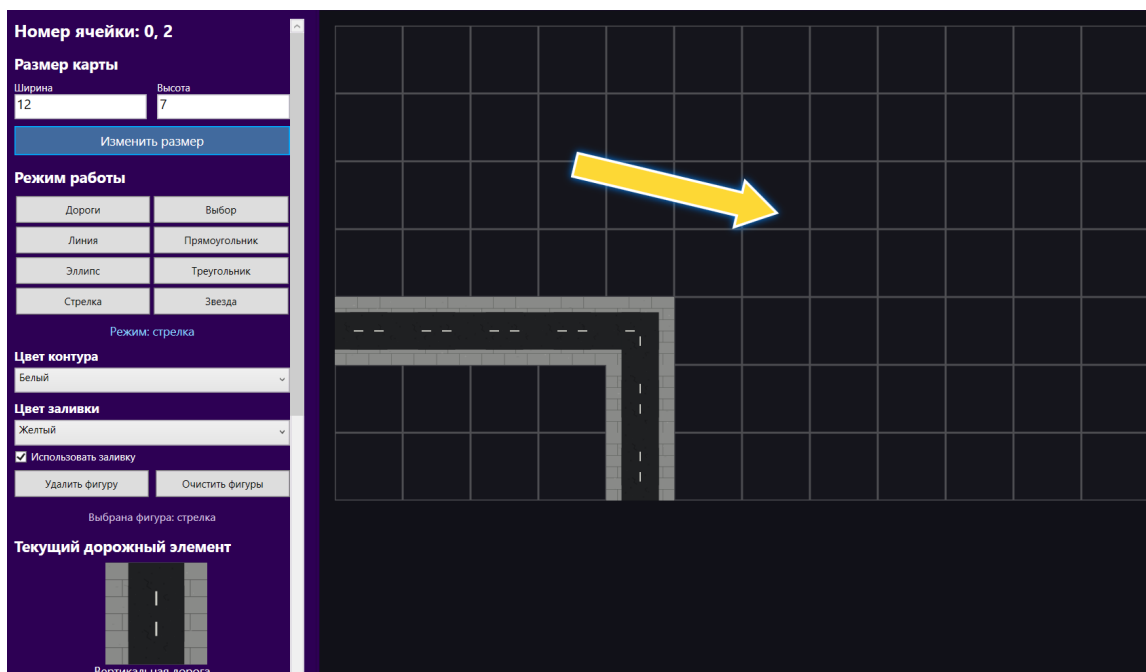


Рис. 9 – Создание стрелки поверх дорожной карты

4.6 Выбор и перемещение фигуры

После переключения в режим «Выбор» фигуру можно выделить и переместить. Выделенный объект подсвечивается, а его тип отображается на боковой панели. Пример перемещения стрелки представлен на рисунке 10.

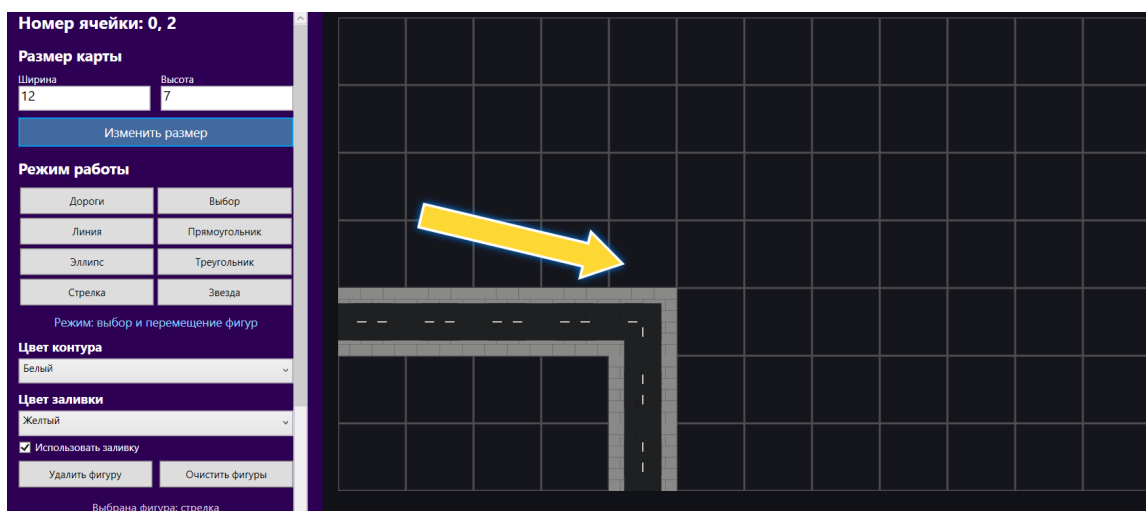


Рис. 10 – Выбор и перемещение графической фигуры

4.7 Экспорт карты в PNG

Команда «Экспорт PNG» открывает диалог сохранения изображения. Пользователь задает имя файла и каталог, как показано на рисунке 11.

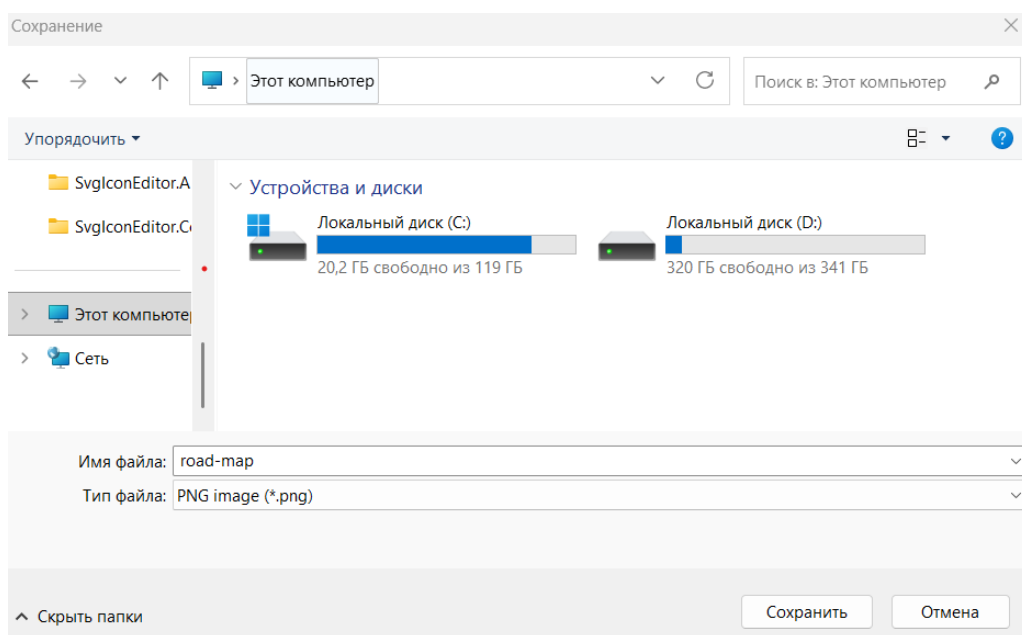


Рис. 11 – Диалог экспорта карты в формат PNG

Итоговый файл включает дорожные элементы, сетку и фигуры дополнительного слоя. Результат экспорта показан на рисунке 12.

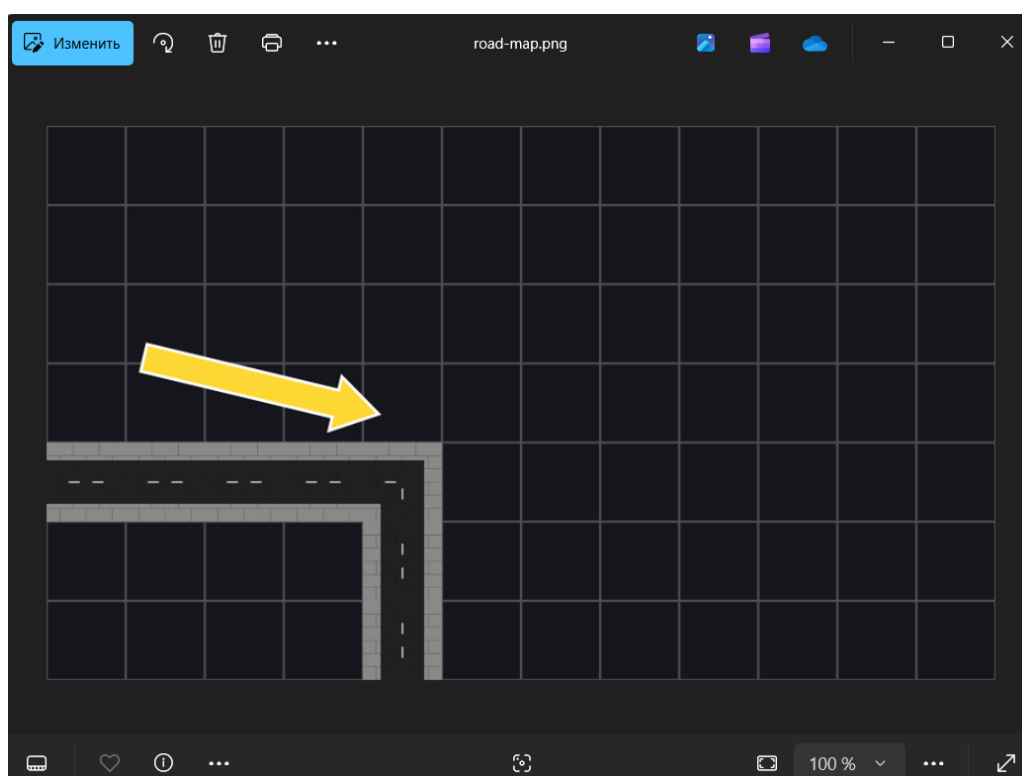


Рис. 12 – Результат экспорта дорожной карты

4.8 Работа с палитрой дорожных элементов

В палитру входят пустая ячейка, тротуарная плитка, горизонтальная и вертикальная дороги, четыре варианта поворотов, четыре Т-образных

развилки и перекресток. Некоторые элементы дополнены пешеходными переходами. Палитра представлена на рисунке 13.

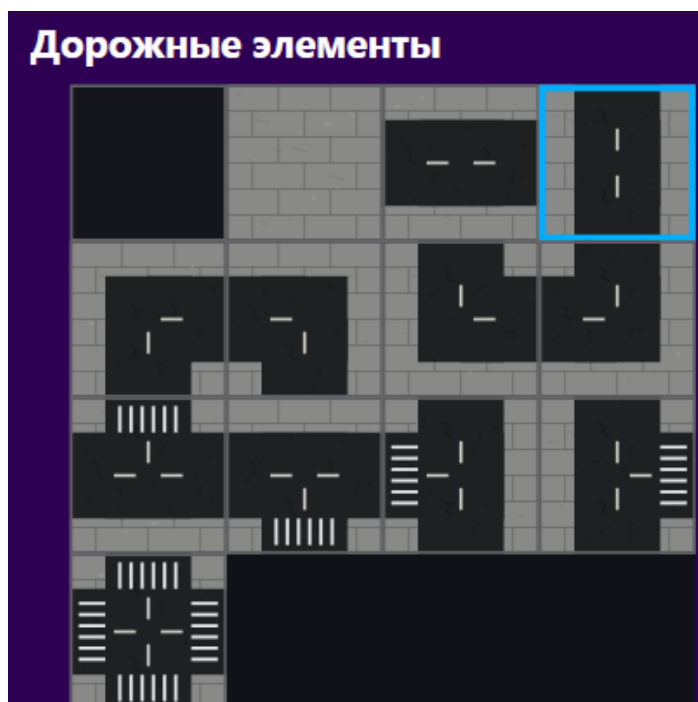


Рис. 13 – Палитра дорожных элементов

4.9 Масштабирование и настройка оформления

На боковой панели расположены кнопки изменения масштаба, информация о текущем значении и элементы настройки цветовой схемы. Синяя тема интерфейса и блок настроек показаны на рисунке 14.

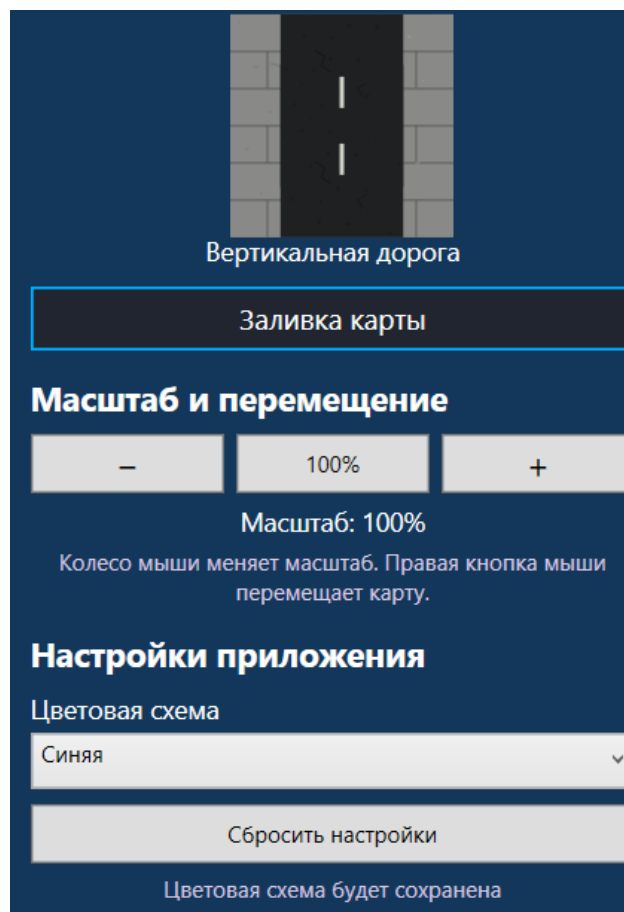


Рис. 14 – Панель масштабирования и пользовательских настроек

Заключение

В ходе выполнения курсового проекта разработано приложение Road Editor, предназначенное для построения и редактирования дорожных карт. Реализована модель карты, основанная на двумерном массиве типизированных элементов, а также графический интерфейс для размещения дорог, поворотов, развилок, перекрестков и тротуарной плитки.

В приложение добавлен отдельный слой графических фигур. Пользователь может рисовать линии, прямоугольники, эллипсы, треугольники, стрелки и звезды, изменять их оформление, выделять, перемещать и удалять. Реализованы отмена и повтор действий, изменение масштаба, перемещение карты, сохранение и загрузка проекта, экспорт результата в PNG, справка и пользовательские настройки.

Разделение решения на проекты RoadEditor.App и RoadEditor.Core позволило отделить основную модель от пользовательского интерфейса. Использование интерфейсов, наследования, коллекций и событий обеспечивает расширяемость программы и соответствует принципам объектно-ориентированного проектирования.

Разработанное приложение может быть расширено дополнительными типами дорог, автоматической проверкой соединения соседних плиток, сохранением нескольких слоев, привязкой фигур к сетке и экспортом в векторные форматы.

Список литературы

1. ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления.
2. Microsoft. Windows Presentation Foundation documentation [Электронный ресурс]. URL: <https://learn.microsoft.com/dotnet/desktop/wpf/> (дата обращения: 16.06.2026).
3. Microsoft. C# documentation [Электронный ресурс]. URL: <https://learn.microsoft.com/dotnet/csharp/> (дата обращения: 16.06.2026).
4. Microsoft. Serialize and deserialize JSON using C# [Электронный ресурс]. URL: <https://learn.microsoft.com/dotnet/standard/serialization/system-text-json/overview> (дата обращения: 16.06.2026).
5. Microsoft. RenderTargetBitmap Class [Электронный ресурс]. URL: <https://learn.microsoft.com/dotnet/api/system.windows.media.imaging.rendertargetbitmap> (дата обращения: 16.06.2026).
6. Microsoft. XAML overview [Электронный ресурс]. URL: <https://learn.microsoft.com/dotnet/desktop/wpf/xaml/> (дата обращения: 16.06.2026).
7. Git. Official documentation [Электронный ресурс]. URL: <https://git-scm.com/docs> (дата обращения: 16.06.2026).